

DESIGN BASELINE

A reusable design and frontend authority for new projects.

Copy this file into a new repository as `DESIGN.md`, fill in the four blanks marked `<...>`, and delete nothing else. Everything not in a blank is a rule, not a suggestion.

When code and this document disagree, this document wins, or the document gets changed first, deliberately, with a reason written into it.

0. How to use this file

Marker	Meaning
MUST	Non-negotiable. A pull request that breaks it does not ship.
SHOULD	Default. Deviating is allowed once, with a comment in the code saying why.
<code><...></code>	Fill in per project.

Four blanks to fill:

1. `<PROJECT>` : the product name.
 2. `<PROMISE>` : one sentence, what a visitor gets, in their words.
 3. `<ACCENT>` : one accent hue (see §4).
 4. `<FACES>` : the four typefaces (see §3).
-

1. Principles

1.1 Proof over claim. Show the thing working. A screenshot, a live widget, a scrubbed film of the real product beats any adjective. Copy supports the proof; it never replaces it.

1.2 Motion answers a gesture. Scroll is the camera. The visitor drives; the page responds. Nothing important moves on its own timer, because a visitor who did not cause a movement cannot predict it and stops trusting the page.

1.3 Legible before beautiful. Every rule in §3 and §4 outranks every rule in §6. A gorgeous page nobody reads has failed. If an effect makes text harder to read, the effect loses. Always.

1.4 One decision, visible everywhere. Two greys that are almost the same, three sans-serifs that are almost the same, two blues 40° apart. These read as indecision, not as richness. Fewer values, used consistently, always look more expensive.

2. The hero: a scroll animation is mandatory

This is the one section every project in this baseline shares.

2.1 The rule

Every project **MUST** ship at least one scroll-driven animation in the hero, and that animation **MUST** be produced with Higgsfield.

Not a fade-in. Not a CSS parallax. A real piece of generated footage or a generated image sequence whose playhead is bound to scroll position, so that scrolling forward advances it and scrolling back rewinds it, frame for frame.

2.2 Why this rule exists

A hero has about **three seconds** to establish that the page is worth reading. Static heroes are indistinguishable from templates. A scroll-bound animation does four things at once that nothing else does:

1. **It proves craft instantly.** The visitor does not have to be told the work is good; their thumb demonstrates it.
2. **It creates a reason to scroll.** The first scroll is the hardest one to earn. If scrolling visibly *does something*, it happens by reflex.
3. **It is deterministic.** Unlike an autoplaying video, a scrubbed one cannot be "already over" when the visitor arrives, and cannot desync.
4. **It is reversible.** Scroll up and the visitor gets the beginning back. Autoplay has no such courtesy.

2.3 Producing the footage with Higgsfield

Brief the shot before generating anything. A scrub animation needs a *single continuous camera move or transformation*, not a cut and not a montage. Cuts are unusable, because a scrubbed cut reads as a glitch.

Good hero shot types, in order of how well they scrub:

Shot	Why it works
Assembly / disassembly , an object coming apart or together	Every frame is visibly different; scrubbing feels like control
Slow orbit around one subject	Continuous, no cuts, loops naturally
Push-in / dolly toward a subject	Maps directly onto "scrolling deeper"
Reveal , a material forming, a light coming on	Clear start and end state

Avoid: anything with a cut, anything with rapid motion (scrubbing fast motion aliases badly), anything where the subject leaves frame.

Generation checklist

- **MUST** generate at 16:9 and, separately, a portrait crop for phones. Letting `object-cover` slice a 16:9 master on a phone throws away the subject.
- **MUST** end on a frame you are happy to hold. The last frame is what the visitor sees for as long as they stop scrolling.
- **SHOULD** be 4–8 seconds of source material. Shorter has no room to breathe; longer means the visitor never reaches the end.
- **SHOULD** be generated against the page's background colour, not a colour "close to" it (see §2.5).

2.4 Encoding for scrubbing

A normal MP4 is encoded for *playback*, not for *seeking*. Seeking into a normal MP4 means decoding from the last keyframe forward, which on a mid-range phone is tens of milliseconds per seek, and a scrub issues one seek per frame. That is the entire reason scrub animations stutter.

MUST encode All-Intra: every frame is a keyframe.

```
ffmpeg -i source.mp4 \  
-c:v libx264 -preset slow -crf 20 \  
-g 1 -keyint_min 1 \  
-x264-params "aq-mode=3:scenecut=0" \  
-pix_fmt yuv420p -movflags +faststart -an \  
hero.mp4
```

- `-g 1 -keyint_min 1`: every frame independently decodable, so each seek is a single-frame decode.
- `-an`: a scrubbed video has no audio. Shipping the track is waste.
- `+faststart`: moves the index to the front so playback can begin before the file has finished downloading.

Budgets

Asset	Budget
Desktop hero film	≤ 6 MB
Phone hero film	≤ 2.5 MB
Poster frame (first frame, JPEG/WebP)	≤ 120 KB

All-Intra roughly triples the bitrate of a normal encode. If you are over budget, reduce **resolution** first (a scrubbed hero at 1408×980 is plenty), then frame rate (24 fps is enough), then duration. Do not raise the CRF above 23. Banding in a dark gradient is very visible.

2.5 Compositing it onto the page

The most common failure is a **visible black rectangle** around the footage. Generated video almost never has a true-black floor; codec noise lifts it to `rgb(8,8,10)` -ish, which is lighter than a `#050505` page.

Two fixes, use both:

1. **Crush the floor at encode time** so the video's black matches the page's black exactly:

```
-vf "curves=all='0/0 0.17/0.010 0.42/0.40 1/1'"
```

2. **Blend it.** `mix-blend-mode: lighten` takes the brighter of video and page, so anything darker than the page *becomes* the page. Prefer `lighten` over `screen`: `screen` *adds*, so residual noise over a large rectangle still lifts the page.

Trap: `mix-blend-mode` composites against its own **stacking context**, not against the page. `position: sticky`, `transform`, `filter`, `opacity < 1`, `will-change` and (in Tailwind v4) the standalone `translate` property each create one. If a blended element sits inside any of those, it blends against that box and you get the black rectangle back. Verify by sampling pixels either side of the seam, not by eye.

2.6 Implementing the scrub

- **MUST** bind the playhead to a scroll progress value in `[0,1]`, and **MUST** be idempotent: the same progress always produces the same frame, so jump-navigation and refresh-mid-page both land correctly.
- **MUST** show the poster image until the first real frame is decoded.
- **MUST** provide a reduced-motion path: `prefers-reduced-motion: reduce` gets the poster frame and no scrubbing.
- **SHOULD** preload only metadata, then the file, once the hero is within ~1.5 viewports.

Trap: calling `play()` to prime a media element whose current position is the **end** of the resource is specified to seek back to the beginning first. If you prime on load and the visitor refreshed mid-page, the hero snaps to frame 0 and freezes. Only prime from a standing start; otherwise apply the wanted position directly.

2.7 Making it smooth

Encoding (§2.4) is necessary and not sufficient. A scrub stutters for seven separate reasons, and fixing six of them still leaves a stuttering hero. Do all seven.

1. **One frame loop for the whole page. MUST.** A smooth-scroll library on its own `requestAnimationFrame`, plus an animation library on another, gives two frame budgets that drift apart. Drive the smooth-scroll from the animation library's ticker and let it push scroll updates:

```
lenis.on('scroll', ScrollTrigger.update)
gsap.ticker.add((t) => lenis.raf(t * 1000))
gsap.ticker.lagSmoothing(0) // never "catch up" after a dropped frame
```

`lagSmoothing(0)` matters: the default behaviour, on a frame that took too long, jumps the timeline forward to compensate. On a scrubbed film that reads as the video skipping.

2. Smooth the input, not the output. MUST. A mouse wheel emits large discrete steps. Feeding those straight into a playhead produces a stair-step no decoder can rescue. A momentum layer (Lenis or equivalent) turns them into a continuous value:

```
new Lenis({
  duration: 1.0,
  easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)),
  wheelMultiplier: 0.75,
  smoothWheel: !prefersReduced,
})
```

This is the opposite of easing the animation itself (§6.2). Smooth the **gesture** going in; never ease the **frame** coming out.

3. Coalesce seeks. MUST. Never issue a seek while one is pending. Keep only the newest target and issue it the moment the previous `seeked` fires, including when the visitor has reversed direction. Uncoalesced seeks queue, and the film ends up seconds behind the thumb.

4. Ignore seeks smaller than half a frame. SHOULD. Most requested seeks land on the frame already showing. Skip them:

```
const SEEK_EPSILON = 1 / 48 // half a frame at 24 fps
if (Math.abs(video.currentTime - next) < SEEK_EPSILON) { draw(); return }
```

This alone removes the majority of seek traffic on a slow scroll.

5. Paint when a frame actually exists, not on the next tick. SHOULD. `requestVideoFrameCallback` fires when a new frame has been decoded and is ready to composite. Painting on `requestAnimationFrame` instead means painting the *previous* frame roughly half the time.

6. Watchdog the seek. MUST. `seeked` is not guaranteed to arrive; on iOS it sometimes does not. Without a timeout, the coalescing lock is never released and the hero freezes for the rest of the visit:

```
watchdog = setTimeout(() => { seeking = false; draw(); drain() }, 280)
```

7. Halve the pixels on phones. MUST. Decode cost scales with pixel count. A separate portrait encode at half the linear resolution is a quarter of the work per seek, and it is the single biggest win on an older phone. It also gives the correct crop instead of letting `object-cover` slice the subject away.

Two more things that decide whether it feels smooth:

- **Paint to a `<canvas>`, not to the `<video>` element**, if the footage needs any masking, filtering or blending. A CSS `filter` on a live video makes the compositor build a fresh filtered layer every frame; doing the same work in a 2D context is one layer and one draw call.
- **Nothing else heavy may run during the scrub**. A WebGL scene, a per-word blur or a parallax on the same screen competes for the same main thread. If the hero scrub must be smooth, it gets the budget alone.

Verify on the worst device you support, by scrubbing. This class of failure is device-specific and does not appear in a screenshot or in a desktop browser. If you cannot test on the device, at minimum throttle CPU 6× in DevTools and scrub the hero end to end, forwards and backwards.

2.8 What else the hero needs

- One headline. One supporting line. **One** primary action.
- The primary action is **filled**, not an outline. It is the single thing to press; it should look it.
- **MUST NOT** run body copy across a busy image. If copy and picture compete for the same pixels, move the picture.

3. Typography

3.1 Four faces, four jobs

Pick by **role**, never by mood. Four is the ceiling.

Role	Job	Character to look for
Poster	Hero headline, full-bleed wordmarks	Heavy, ideally condensed so long words fit one line
Display	Section headings	A real weight axis, so h2 and h3 differ by weight not only size
Body	Paragraphs, UI, buttons, nav	Drawn for screens: large x-height, open apertures, unambiguous <code>I l 1</code>
Label	Kickers, metadata, captions	Narrow, works uppercase, legible at 12–15px

`<FACES>` : record the four here, with the CSS variable name for each.

Rules

- **MUST NOT** ship three faces from the same genre. Three grotesques that are nearly alike give the page no rank; the reader cannot tell what is important.
- **MUST NOT** request a weight the face does not ship. The browser fakes it and the edges smear. Check the axis before writing `font-bold`.

- **MUST** name the variables by role (`--font-body-face`), never by vendor (`--font-geist-sans`). Faces get replaced; roles do not.
- Reserve monospace for **actual code**. Using it for labels makes a marketing page look like a terminal prop.

3.2 The reading scale

These are floors, measured on the rendered page, not intentions.

Element	Size	Colour on dark	Measure
Hero lead	18–19 px	100 % foreground	34–46 ch
Section lead	17–18 px	80–85 %	46–52 ch
Card body	16 px	78 %	38–48 ch
Meta, captions	14–15 px	65–75 %	n/a
Footnote (one line)	13 px	60–65 %	n/a

- **MUST NOT** ship body copy below **16 px**.
- **MUST NOT** exceed **75 characters per line**. Past that the eye loses the start of the next line. Set it with `max-w-[48ch]`, not with `rem`. `ch` is the only unit that means what you want here.
- **MUST** set line height by role: 1.6 for reading text, 0.9–1.05 for poster headlines.
- **SHOULD** range copy **left**. Centred text forces the eye to re-find the start of every line; acceptable for one or two lines, wrong for a paragraph.

Verify, do not assume. Measure the built page:

```
// widest rendered line of a leaf text block, in characters
const r = document.createRange(); r.selectNodeContents(el)
const w = Math.max(...r.getClientRects()).map(x => x.width)
const ch = w / (parseFloat(getComputedStyle(el).fontSize) * 0.5)
```

Measuring the element's own width instead gives false results for grid rows and flex containers.

4. Colour

4.1 One accent family

- **MUST** ship exactly **one** accent hue, plus an optional neighbour within ~15° for support. Two accents 40–60° apart are the classic failure: too similar to read as a deliberate pair, too different to read as one family, so nothing is "the" brand colour.

- **SHOULD** define a ramp of that one hue rather than reach for a second: a tint (borders, glows), a soft (secondary text), a base (the accent), a deep (fills).
- Use `oklch()`. Two colours with the same `L` in `oklch` actually look equally bright, which is not true in HSL.

`<ACCENT>` : record the hue and the four ramp steps here.

4.2 Contrast is arithmetic, not taste

Compute it; do not eyeball it. Against a `#050505` page:

Lightness <code>L*</code>	Contrast on black	Contrast on white
96 (near-white)	19 : 1	20 : 1
67 (a mid accent)	8 : 1	2.5 : 1
56 (a "muted" grey)	5.6 : 1	3.7 : 1

Minimums: 4.5 : 1 for body copy, 3 : 1 for large text (≥ 24 px or ≥ 19 px bold) and for UI borders that carry meaning.

Two consequences people get wrong:

- **Inverting a dark page to light does not improve contrast.** White on near-black is already $\sim 19 : 1$; black on white is $\sim 20 : 1$. What inverting *does* do is break every accent, because accents mixed for a dark canvas land at 2.5–3 : 1 on white. If a project needs both, it needs **two** accent values, and the light one must be mixed separately.
- **A "muted" grey is a decision, not a default.** `--muted-foreground` at 5.6 : 1 is legal and still hard to read at 14 px. Prefer `foreground / 78 %` gives the same visual softness, far more contrast.

4.3 Dark and light surfaces

- One canvas per project. **SHOULD** be dark if the product has glow, 3D or video; light if the product is mostly text.
- A single inverted section is a strong device, the one lit room in a dark house, but the fixed chrome (nav, wordmark, scrim) is styled for the other surface. **MUST** flip the chrome for exactly the stretch the inverted surface is under it, and flip it back.

5. Layout and spacing

- **One spacing scale**, powers of a 4 px base: 4, 8, 12, 16, 24, 32, 48, 64, 96, 128. **MUST NOT** introduce values off the scale.
- Section rhythm: vertical padding **SHOULD** be at least 1.5× the largest gap inside the section, so sections read as separate rooms.

- **MUST** cap the content column. Full-bleed body copy on a 1440 px screen is unreadable regardless of font size (see §3.2).
- **MUST NOT** let anything cause horizontal overflow. Assert it: `document.documentElement.scrollWidth === clientWidth` at 320, 390, 768, 1024, 1440 and 1920.
- Anchor targets **MUST** carry `scroll-margin-top` clearing the fixed chrome, or a pasted deep link lands under the nav.

6. Motion

6.1 The opacity floor: the rule people break most

MUST NOT animate text from `opacity: 0`.

An element fading in from zero is, by definition, unreadable for the length of the fade. On a fast connection nobody notices. On a phone over mobile data, that is a visible stretch in which the page appears broken, and it is the single most common cause of "I can't read anything on your site."

- Text entrances **MUST** start at ≥ 0.45 opacity and rise.
- Prefer moving `transform` and letting opacity do very little.
- Decorative, non-text elements may fade from 0.

6.2 Duration and easing

Kind	Duration	Easing
Hover, focus, small state	120–180 ms	<code>ease-out</code>
Entrance, reveal	300–500 ms	<code>cubic-bezier(0.22, 1, 0.36, 1)</code>
Scroll-scrubbed	n/a, bound to scroll	linear

- **MUST NOT** ease a scrubbed animation. The visitor's thumb is the easing curve; adding another makes the page feel like it is lagging.
- Stagger **SHOULD** be ≤ 40 ms per item and cap out at ~8 items.

6.3 Respecting the visitor

- **MUST** honour `prefers-reduced-motion: reduce`: no scrub, no parallax, no auto-motion. The reduced path must still make sense as a page.
- **SHOULD** measure a few seconds of frame timing after load and degrade on weak devices: drop per-element blur, cut particle counts, disable parallax. A slide-show at 15 fps is worse than no animation.
- Animate `transform`, `opacity` and `filter` only. Animating layout properties (`width`, `top`, `margin`) forces reflow every frame.

7. Media and performance

Budgets, measured on a cold load of the first screen:

	Budget
JS on first load	≤ 200 KB compressed
Fonts	≤ 4 families, subset to the scripts used
Largest Contentful Paint	≤ 2.5 s on a mid-range phone, 4G
Anything below the fold	0 bytes until near-viewport

- **MUST** lazy-load heavy scenes and films behind a near-viewport check, with a generous margin (~1.5 viewports). Too tight and the visitor arrives at an empty frame; the work has to *finish* before they get there, not start.
- **MUST NOT** let two large media files download in parallel on the first screen. They halve each other's bandwidth and both arrive late.
- **MUST** give lazy content a visible placeholder. An empty bordered box reads as a bug.
- Images: modern format, explicit `width / height` to reserve the box, `loading="lazy"` below the fold.
- If the project uses WebGL: **MUST** handle `webglcontextlost`. Browsers cap concurrent contexts; when a page has several scenes, one of them will eventually go black without a handler.

8. Accessibility

- **MUST** keep a real DOM heading order: one `h1`, then `h2`, then `h3`, with no levels skipped for styling.
- Animated or decorated text **MUST** still be one readable string for a screen reader, usually a visually-hidden copy of the full sentence alongside the decorated, `aria-hidden` version.
- Every interactive element **MUST** have a visible focus ring. Never `outline: none` without a replacement.
- Touch targets **MUST** be $\geq 44 \times 44$ px.
- Colour **MUST NOT** be the only carrier of meaning; pair it with text, weight, or an icon.
- Keyboard: the whole page must be operable without a pointer, and a modal must trap focus and close on `Escape`.

9. Content

Write for the person paying, not for the person building.

- **MUST NOT** use internal vocabulary in customer-facing copy. "Robust backend", "clean technical foundation", "full-stack", "scalable architecture" describe how the work feels to you, not what the buyer gets.
- Answer, in this order: **what you get, what it costs, how long it takes, what you have to do**. A page that skips these is a brochure, not an offer.
- One idea per sentence. If a sentence needs a comma to hold two clauses together, it is probably two sentences.
- **MUST NOT** claim a feature that does not exist. A promised dashboard, payout schedule or SLA with nothing behind it is the fastest way to lose the first customer who says yes.
- Strip em-dashes and en-dashes from generated copy. They are a tell, and a comma or a full stop is nearly always better.
- One CTA per screen. Two competing buttons halve both.

Portfolio ≠ landing page. A portfolio impresses; a landing page explains and converts. Their tones are in direct conflict, and a page attempting both reads as "too complicated". If a project needs both, give them separate routes, and separate domains if the audiences differ.

10. Legal and consent baseline

Ship these from day one; retrofitting is worse.

- **Imprint and privacy pages**, reachable from every route including standalone landing pages.
 - **Consent before loading anything optional.** Analytics, embedded agents, third-party fonts and maps do not load until allowed. Store the decision **versioned** and **dated**, with each category independently answerable. A stored record must be able to say which questions were actually asked.
 - The privacy page **MUST** describe every processing the site actually performs, including any feature that collects a *third party's* data (referral programmes, forms that take someone else's contact details).
 - `sitemap.xml` and `robots.txt` **MUST** list every real route. Adding a route without adding it to the sitemap is the most common SEO own-goal.
 - Never put a secret in client code. Anything `NEXT_PUBLIC_*`, any data attribute, any inline script is publicly readable.
-

11. Known traps

Each of these cost real debugging time. They repeat across projects.

Stacking contexts. `transform`, `filter`, `opacity < 1`, `will-change`, `position: sticky` and (Tailwind v4) the standalone `translate:` property each create one. They break `mix-blend-mode`, `position: fixed` children, and z-index reasoning. When something blends or layers wrongly, look up the ancestor chain for one of these first.

Tailwind v4 emits `translate:` as its own property, separate from `transform`. Writing `transform: translate(-50%, -50%) scale(k)` in JS while a utility class emits `translate: -50% -50%` applies the shift twice. Scale in `transform`; position with the utility, or with neither.

Trailing whitespace inside `inline-block` is trimmed. Splitting a sentence into per-word spans welds it into one unbroken string. The space must live *between* the spans.

CSS `perspective`. An element translated to or past the perspective distance is behind the viewer's eye and is not painted at all. Cap depth translations well below the perspective value, or elements pop into existence instead of approaching.

`play()` **on a media element positioned at the end of the resource** is specified to rewind to the start first. See §2.6.

`page.mouse.click()` **in tests does not emit a prior `pointermove`**. Components that arm on hover will not respond, and the test reports a false negative. Hover, then press, then release.

A stale dev server serves a stale build. If a fix "does not work", confirm the server actually restarted before debugging the fix. Launch on a fresh port when in doubt.

12. Anti-patterns

Don't	Because
Fade text in from <code>opacity: 0</code>	Unreadable for the length of the fade (§6.1)
Body copy under 16 px	Fails on the devices most people use
Lines over 75 characters	The eye loses the next line's start
Three near-identical sans faces	No rank; nothing looks important
Two accents 40–60° apart	Reads as indecision, not richness
A gradient clipped to text	Fragile in WebKit; contrast varies across the word
Outline button as the only CTA	The weakest possible primary action
Centring a paragraph	Every line starts somewhere new
<code>mailto:</code> as the only conversion path	Dead on any device without a mail client; no record, no source
Autoplaying video in a hero	Already over when the visitor arrives
A spinner where content will be	Reads as broken; use a shaped placeholder
Copy over a busy image	Neither wins; move the image

13. Ship checklist

Run every line. "Looks fine" is not a check.

Content

- ☐ Every claim on the page is true today.
- ☐ Price, duration and next step are answered somewhere.
- ☐ No internal vocabulary in customer-facing copy.

Typography

- ☐ No text block below 16 px (measured, §3.2).
- ☐ No line over 75 characters at 1440 px (measured, §3.2).
- ☐ No faked font weights.

Colour

- ☐ Body copy $\geq 4.5 : 1$, large text and meaningful borders $\geq 3 : 1$.
- ☐ One accent family.

Motion

- ☐ Hero scroll animation present, scrubbed, reversible (§2).
- ☐ All seven smoothness rules applied, verified by scrubbing on the slowest supported device (§2.7).
- ☐ No text entrance starting below 0.45 opacity.
- ☐ `prefers-reduced-motion` path verified.

Layout

- ☐ Zero horizontal overflow at 320 / 390 / 768 / 1024 / 1440 / 1920.
- ☐ Anchor links land clear of the fixed chrome.

Performance

- ☐ Hero film within budget (§2.4); poster shows before it.
- ☐ Nothing below the fold loads on first paint.
- ☐ LCP ≤ 2.5 s on a mid-range phone.

Accessibility

- ☐ Heading order intact; decorated text has a readable equivalent.
- ☐ Visible focus on every control; full keyboard operation.
- ☐ Touch targets ≥ 44 px.

Legal

- ☐ Imprint and privacy reachable from every route.
- ☐ Consent gates every optional load; record versioned and dated.
- ☐ Sitemap lists every route.

14. Working agreement

1. **Verify, do not assert.** Screenshot it, measure it, sample the pixels. "Should work" is not a result.
2. **One change, one reason.** If you cannot write the reason into the commit, the change is not ready.
3. **Comment the non-obvious.** Every trap in §11 was invisible in the diff that caused it. A sentence saying *why* a value is what it is saves the next person a day.
4. **Update this file when reality changes.** A design authority nobody trusts is worse than none.